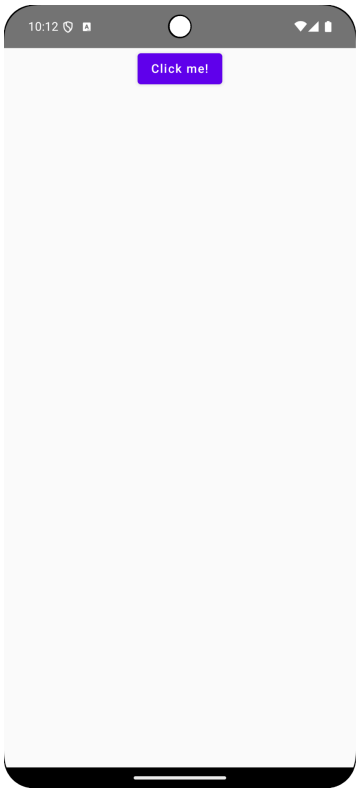
Practical Instructions

Practical 1: Splash Screens

1.1 Add a static splash screen to the Android app

Prestep

1. In Android Studio, navigate to File -> Open. In the file dialog, navigate to your local repository and then to the subdirectory /practical1/starter. The project should be compilable and runnable and will look like this.



Add the splash screen compatibility library

1. Navigate to the `androidApp/src/build.gradle.kts file`.
2. Add the splash screen library dependency.. It should look something like this:

```
dependencies {
    implementation("androidx.core:core-splashscreen:1.2.0")
```

```
    …
}
```

3. Remember to sync Gradle.

Import the .svg file as a vector asset

1. In Android Studio's menu bar, navigate to `File > New > Vector Asset`.
2. In the `Configure Vector Asset` dialog, select the `Local File` radio button.
3. In the `Path file` selector control, navigate to the `camera.svg` file (in your local repository) under `assets/practical1/android` and click on the Open button.
4. Set the size of the vector asset to be 100 dp x100 dp.
5. Click on the Next button.
6. In the `Confirm Icon Path` dialog, ensure that the source set (`main`) and output file (`res/drawable/camera.xml`) are set correctly, and click on the `Finish` button.
7. In the `Project` navigator window, ensure the corresponding XML file appears in the output directory as above.

Make the vector drawable fit

The Android splash screen API has some dimension requirements for the splash screen icon:

- Icon with an icon background: must be 240×240 dp and fit within a circle 160 dp in diameter.
- Icon without an icon background: must be 288×288 dp and fit within a circle 192 dp in diameter.

This is because the splash screen API applies a circle mask to the screen. Conforming to the above requirements ensures that the entire logo fits into the circle mask's visible area. You can read more about this here.

An easy way to do this is as follows:

1. Open the vector drawable XML file, and note the viewport width (`android:viewportWidth`) and height (`android:viewportHeight`)
2. Wrap all <`path`> elements into a top-level <`group`> element with the following attributes:
    a. `android:pivotX="x"` where x is the value is half `android:viewportWidth` determined in step 1 (without dp)
    b. `android:pivotY="y"` where y is the value is half `android:viewportHeight` determined in step 2 (without dp)
    c. `android:scaleX="0.4"` (this value is based on the width of your vector asset, mine was 100dp)
    d. `android:scaleY="0.4"` (this value is based on the height of your vector asset, mine was 100dp)

Create a splash screen theme

It's now necessary to create a customized splash screen theme:

1. Right-click on `androidApp/src/main/res/values`
2. Select `New > Values resource file` in the menu
3. Call the new values resource file something like "`splash_screen_theme.xml`" and click on the OK button
4. Edit the resulting file to look as follows:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <style name="Theme.App.Splash" parent="Theme.SplashScreen">
        <item name="windowSplashScreenBackground">
            #E24462
        </item>
        <item name="windowSplashScreenAnimatedIcon">
            @drawable/camera
        </item>
        <item name="postSplashScreenTheme">
            @android:style/Theme.Material.Light.NoActionBar
        </item>
    </style>
</resources>
```

Let's analyze our new splash screen theme. The splash screen theme, `Theme.App.Splash`, inherits from the `Theme.SplashScreen` theme defined by the Android splash screen library. We are overriding three of its attributes:

`windowSplashScreenAnimatedIcon` to define the vector drawable acting as our logo

`windowSplashScreenBackground` to define the background color of the rest of the screen

`postSplashScreenTheme` to define the next theme - this will most probably be the theme of the rest of the application

Apply the splash screen theme

1. Navigate to the `androidApp/src/main/res/AndroidManifest.xml` file

2. Change the theme of both the application and the first `Activity` to `Theme.App.Splash`.

This could look as follows. `MainActivity` is the only `Activity` in the app.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <application
        …
        android:theme="@style/Theme.App.Splash">
        <activity
            …
            android:name=".MainActivity"
            android:theme="@style/Theme.App.Splash">
            …
        </activity>
    </application>
</manifest>
```

Install the splash screen

1. Open up the first activity in the application. In my example app, this is `MainActivity`.
2. Add a call to `installSplashScreen()` before calling `enableEdgeToEdge()`
3. Import the function as the IDE suggests.

Run the application

1. Select `androidApp` in the `Run Configurations` menu, then click on the `Run` button.
2. The splash screen with the logo will show briefly, then disappear to show the first activity content.

Troubleshooting

It might be necessary to change the scaling in the section Making the vector drawable fit to ensure no part of the icon is cut off.

1.2 Add a static splash screen to the iOS app

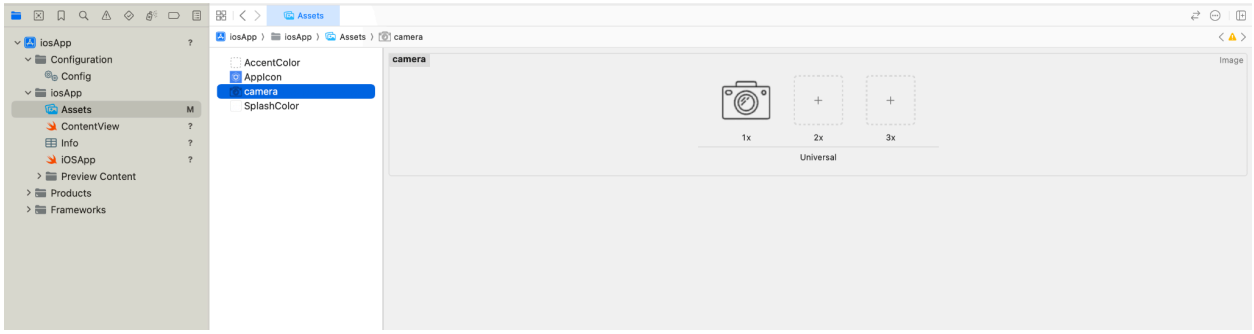
Hop over to Xcode

1. In Android Studio, navigate to `File > Open Project in Xcode`

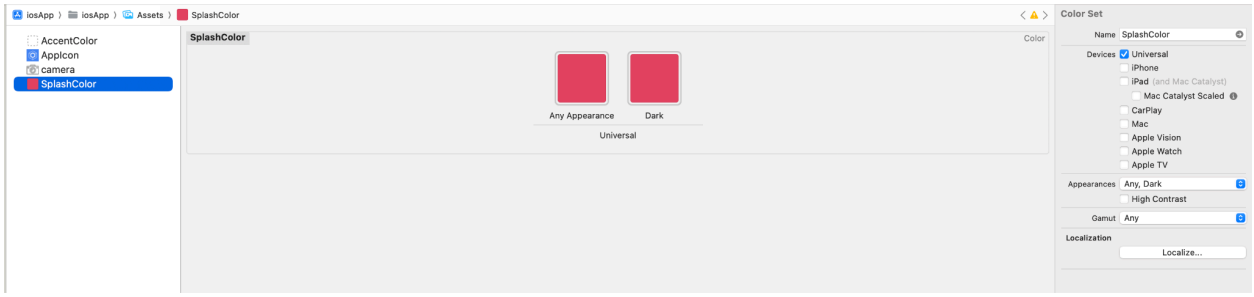
The project will now be correctly opened in Xcode.

Add the image and color to your Xcode project

1. In Xcode, navigate to `iosApp > iosApp` and open up `Assets`
2. Drag and drop your logo from `/assets/practical1/ios` into the white Assets view.

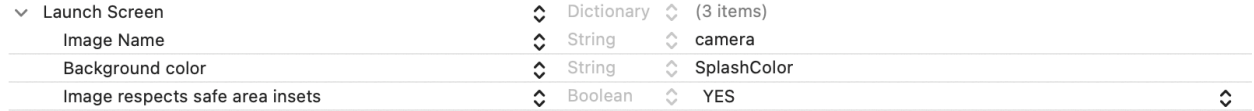


3. Right-click on the `Assets` view and select `New Color Set.`
4. Open the `Attributes inspector`. You may have to open the `Inspectors` panel by opening the button on the top right button  , then  .
5. Click on `Any Appearance` in the `Color` panel.
6. Fill in `#E24462` in the right-side panel's `Hex` value and set the `Name` to `SplashColor`
7. Repeat for the `Dark` appearance.



Create the Launch Screen

1. In the left side panel, navigate and click on the top-level `iosApp` (and make sure to be on the `iosApp` target)
2. In the middle panel, select `Info`
3. In the `Customized iOS Target Properties` table edit the property `Launch Screen`



4. Add three properties underneath `Launch Screen` (there is autocomplete) using right-click-> Add row:
    a. `Image Name`, which is the image you added in the previous step, without the extension.
    b. `Image respects safe area insets`, which is whether to respect safe areas.
    c. `Background color`, which is `SplashColor`.

Run the iOS app to see the launch screen

If you have run the app before, there may have been some caching issues and the launch screen won't show. To fix this, a combination of the following might be necessary:

- Navigate to the menu `Product > Clean Build Folder`
- Hard-close the app already running. You need to swipe up from the bottom, and then close the specific app by swiping it up.
- Delete the app from the device.

You can then run the app as usual and enjoy the very brief showing of the launch screen.

1.3 Add a static splash screen to the Desktop app

1. In Android Studio, add the `camera.png` image from `assets/practical1/desktop` to `src/main/assets/common` in `desktopApp` (you will have to create the directories).
2. Update the `build.gradle.kts` file for `compose.desktop`:

```
compose.desktop {
   application {
       …
       nativeDistributions {
           targetFormats(TargetFormat.Dmg, TargetFormat.Msi,
TargetFormat.Deb)
           …
           appResourcesRootDir =
           layout.projectDirectory.dir("src/main/assets")
                       jvmArgs +=
           "-splash:${'$'}APPDIR/resources/camera.png"
       }
   }
}
```

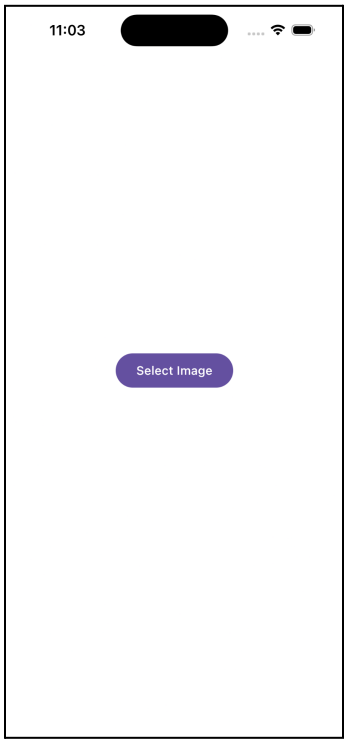
3. Remember to sync Gradle. You may have to run `./gradle clean`.
4. In the terminal: `./gradlew desktopApp:runDistributable`

Practical 2: Debugging KMP shared code from Swift

Option 1: Easy - lldb only

Prestep

1. In Android Studio, navigate to File -> Open. In the file dialog, navigate to your local repository and then to the subdirectory /practical2/starter. The project should be compilable and runnable and will look like this or the permissions screen (if you run it on iOS) (doesn't require an iPhone).



Running the application

1. Run the iOS application on the simulator.
2. Select an image from the gallery.
3. Click on the Blur button.
4. Notice that the image is not being blurred. Let's find out why!

In the next few steps, we will connect to a simulator process to debug it with lldb.

Getting the app process port number

1. Switch to the command line.
2. One way to get the port number is to use the following command:
   `xcrun simctl launch booted com.jetbrains.cameraapp.KotlinConfCameraApp`

   The following output will then be output, with the number at the end being the port number.
   `com.jetbrains.cameraapp.KotlinConfCameraApp: 23535`

Troubleshooting: If a new simulator is created and booted, don't panic! Just install the application on **that** simulator, by dragging the application from this path onto the simulator screen.
The path will be:
`deep-dive-into-kmp/practical2/starter/build/ios/Debug-iphonesimulator/KotlinConfCameraApp.app`
Run the `xcrun` command again to get the port number.

Launching lldb

1. Still in the command line, run the command: `lldb`
2. The `(lldb)` prompt will be displayed.

Attaching to the process

1. In the lldb prompt, use: `process attach -p port_number` (for me it was 23535).
2. It will take a little time, then say something like:
   `Target 0: (KotlinConfCameraApp) stopped.`

Importing to konan_lldb.py pretty printer

1. Let's start by finding this file's path on your machine. Mine is:
   `/Users/pamelahill/.konan/kotlin-native-prebuilt-macos-aarch64-2.3.21/tools/konan_lldb.py`
   Where `pamelahill` is my username, `macos-aarch64` is my architecture, and `2.3.21` is my project's version of Kotlin. Try to find yours by browsing your filesystem.
2. Now run in the lldb prompt:
   `command script import your_path_to_konan_lldb.py`
   I didn't get any feedback when I run it, so don't worry if that's the case.

If you see memory address when you try to print out variables later in lldb, this command wasn't run correctly, maybe check the path / ask for help.

Setting a breakpoint

1. In the lldb prompt:
   `b -f GaussianBlurFilter.ios.kt -l 11`

   This sets a breakpoint on the file `GaussianBlurFilter.ios.kt`, line number 11.

   It will say something like this:
   ```
   Breakpoint 1: where =
   KotlinConfCameraApp.debug.dylib`kfun:com.jetbrains.cameraapp.filt
   er.IOSGaussianBlurFilter#filter(kotlin.String;kotlin.Float){} +
   728 at GaussianBlurFilter.ios.kt:11:5, address =
   0x000000010493ab14
   ```
2. Type: `continue`

   This continues all threads until the next breakpoint is hit. It will say something like this, based on your port number:
   ```
   Process 23535 resuming
   ```

Running to the breakpoint

1. In the app on the simulator, select an image.
2. Click the blur button, and you will see something like this:
   ```
   Process 23535 stopped
   * thread #13, queue = 'com.apple.root.default-qos', stop reason =
   breakpoint 1.1
       frame #0: 0x000000010493ab14
   KotlinConfCameraApp.debug.dylib`kfun:com.jetbrains.cameraapp.filt
   er.IOSGaussianBlurFilter#filter(_this=[],
   imagePath=/Users/pamelahill/Library/Developer/CoreSimulator/Devic
   es/E2CDBC51-474D-49C9-9EDD-C101335B4A44/data/Containers/Data/Appl
   ication/F728F6AD-D9BC-4836-ABC9-7A335A1D559E/tmp/CPH-2023-slides-
   0F48FFAA-7CB3-4782-9EC5-6CB00DFB84E3.jpeg, radius=25){} at
   GaussianBlurFilter.ios.kt:11:5
      8
      9       @OptIn(ExperimentalForeignApi::class)
      10      class IOSGaussianBlurFilter : GaussianBlurFilter {
   ```

```
-> 11          override fun filter(imagePath: String, radius:
Float) {
   12              // Validate file exists and is an image
   13              validateImageFile(imagePath)
   14
Target 0: (KotlinConfCameraApp) stopped.
```

3. The arrow is pointing to where in the code the debugger has stopped.
4. Next, type: `step`

   The arrow will step into the function and advance.

Debugging the blur filter

1. Using the following commands, navigate to line 19. Can you see the error? Can you fix it?

   `n` to step over
   `p` to print out a variable or expression

   You're welcome to use any lldb commands you want, of course. You can either use the `help` command or the slides for a quick guide.

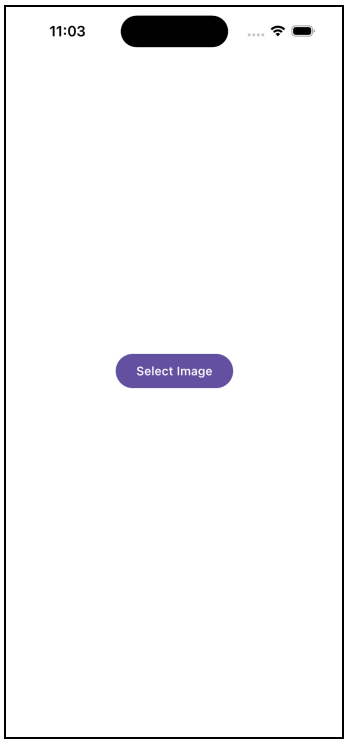
   The solution is in the footnotes of this page.[1]

Option 2: Tricky - use any method of debugging

Prestep

1. In Android Studio, navigate to File -> Open. In the file dialog, navigate to your local repository and then to the subdirectory /practical2/starter. The project should be compilable and runnable and will look like this or the permissions screen (if you run it on iOS) (doesn't require an iPhone).

[1] You need to use `imagePath` to set up the URL not `imagepath`, a variable declared in `App.kt`. On the last page of this document, there are instructions to get global variables in lldb.



Running the application

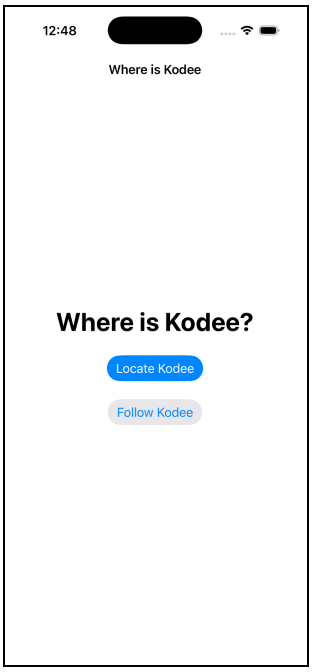
2. Run the iOS application on the simulator.
3. Select an image from the gallery.
4. Click on the Black & White button - the image will be filtered.
5. Click on the Back button and select another image from the gallery.
6. Click on the Black & White button - the image will **not** be filtered. Can you figure out why?

Practical 3: From Objective-C bridge to Swift Export with Kodee

Presteps:

1. In Android Studio, navigate to File -> Open. In the file dialog, navigate to your local repository and then to the subdirectory /practical3/starter.
2. Open the project in Xcode by navigating to File > Open project in Xcode.
3. Run the project (I had trouble for some reason running this project in Android Studio).

The project should be compilable and runnable and will look like this.



Setting up project for Swift Export

1. In Android Studio, navigate and open `gradle > libs.versions.toml`.
2. Change the version of Kotlin to `2.4.0-Beta2`. The latest version of Swift Export is available in this version.
3. Navigate to `shared > build.gradle.kts` and add the following code block in the `kotlin` block configuration:

```
@OptIn(ExperimentalSwiftExportDsl::class)
swiftExport {
    // Root module name
    moduleName = "Shared"

    // Collapse rule
    flattenPackage = "com.jetbrains.whereiskodee.whereiskodee"
}
```

   This configures the shared module to be converted to a Swift module called Shared.
4. Sync gradle.
5. Open the project in Xcode by navigating to `File > Open project in Xcode`.
6. In Xcode, click on the `iosApp` tree item in the Project navigator.
7. Select `iosApp` in `Targets` in the panel that opens.
8. In the middle panel, select `Build Phases`.
9. Expand `Compile Kotlin Framework`.

10. Replace the gradle call in the shell script with the following:

```
./gradlew :shared:embedSwiftExportForXcode
```

This changes the task Xcode invokes to embed the framework when building the app.

Updating the shared Kotlin code

Our Kotlin code no longer needs to use the wrappers to support cancellation and type safety. Let's remove them, and convert the code back to standard Kotlin.

1. In Android Studio, navigate to `shared > iosMain > kotlin > com.jetbrains.whereiskodee.whereiskodee` and delete `FlowWrapper` and `SuspendWrapper` files.
2. Open the file `KodeeSDK`.
3. Change the `locateKodeeWrapper` function to the following:

```
suspend fun locateKodee() = repository.locateKodee()
```

4. Change the `locationHistoryWrapper` function to the following:

```
fun locationHistory() = repository.locationHistory()
```

Updating the Swift code - suspend function

1. Back in Xcode, navigate to the `LocateKodeeViewModel` file.
2. Change the `locate` function code to use Swift Export to call the suspend function. We will also add a little error handling here with a do/try/catch.

```
@MainActor
func locate() async {
    state = .loading

    do {
       let location = try await sdk.locateKodee()
       state = .result(location)
    } catch {
       print("Task was cancelled")
    }
}
```

Notice the following code changes:

a. We have marked the `locate` function with `@MainActor` so that the function contents will run on the “main” thread, not a background thread.
b. We have also marked the `locate` function with the `async` keyword to indicate the function supports concurrency.
c. We need to `await` the result of the `locateKodee` function call as it is marked `async` in the Swift Export code.

3. We need to remove the “old” way of calling the `locate` function. Remove the `locate`() function call from the `init`() function.
4. We now need add the “new” way of calling the function. Navigate to the `LocateKodeeView` file, and change the `Group` to have the following `task` configuration at the bottom of its configuration. When the view appears, a `Task` will be created and the viewModel’s code will run. When the view disappears, the same `Task` will be cancelled, and so will the suspending function’s work.

```
.navigationBarTitleDisplayMode(.inline)
.task {
       await viewModel.locate()
}
```

5. We also need to update the “Locate again” calling code. Find the “Locate again” button in the same file and use the following code as the action of the button

```
Button("Locate again") {
    Task {
        await viewModel.locate()
    }
}
```

This is not the right way to do things, as we are not holding onto a Task handle to cancel when we need to (maybe when the screen disappears?). This also means the suspending function’s work will not get cancelled. But it will do for a demo for now.

Updating the shared Kotlin code - Flow

1. Still in Xcode, navigate to the `FollowKodeeViewModel` file.
2. Change the `startObserving` function code to use Swift Export to call the flow.

```
@MainActor
func startObserving() async {
   do {
       let sequence = sdk.locationHistory().asAsyncSequence()
```

```
        for try await location in sequence {
           self.locations.insert(location, at: 0)
        }
    } catch {
        print("Flow was cancelled")
    }
}
```

Note the following: We need to use the `asAsyncSequence` function on `locationHistory`()'s result. `AsyncSequence` is a Swift concept that can asynchronously wait for a sequence of values. As each arrives, we add the new location to the top of the list.

3. Remove the `startObserving` function call in the `init` function. It will be moved to the view.
4. We now need to update the code in the `FollowKodeeView` file to call the viewModel. Add the following code at the bottom of the `List` configuration code.

```
.navigationBarTitleDisplayMode(.inline)
.task {
    await viewModel.startObserving()
}
```

Importing Combine

1. We also need to import `Combine` in the two **view model**s' files, so that the UI's observability compiles and works correctly.

```
import Shared
import Combine
```

Run the app

1. From Xcode, run the application using the "play" button. I had trouble running a converted app in Android Studio (perhaps something was cached)?

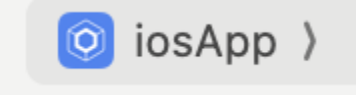
Practical 4: Fixing a Memory Leak

Presteps:

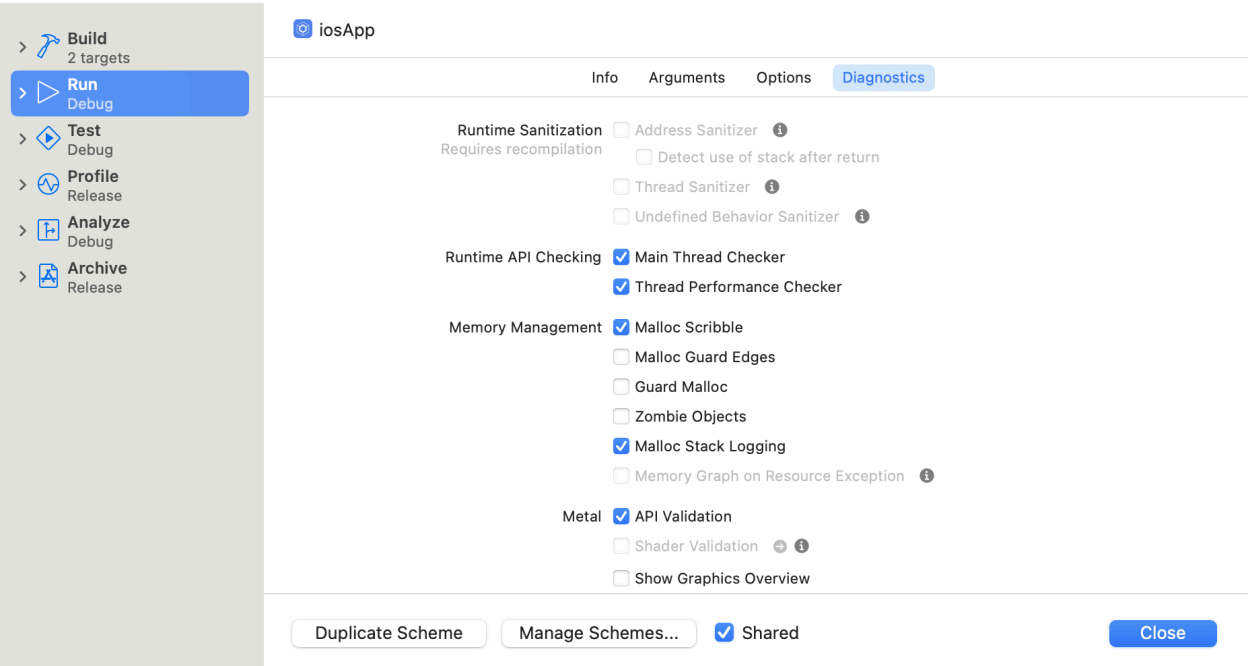
1. In Android Studio, navigate to `File > Open`. In the file dialog, navigate to your local repository and then to the subdirectory `/practical4/starter`. The project should be compilable and runnable.

Enable malloc debugging info on the scheme

1. In Android Studio, open the project in Xcode by navigating to `File > Open project in Xcode`.
2. In Xcode, click on `iosApp` to open the scheme menu. It's at the top of the screen, next to the selected device.

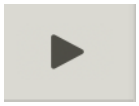


3. Select `Edit Scheme`
4. In the `Memory Management` section of the popup dialog, enable `Malloc Scribble` and `Malloc Stack Logging`, and click the `Close` button.

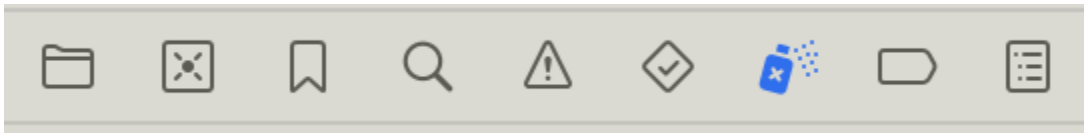


Run and create the leak

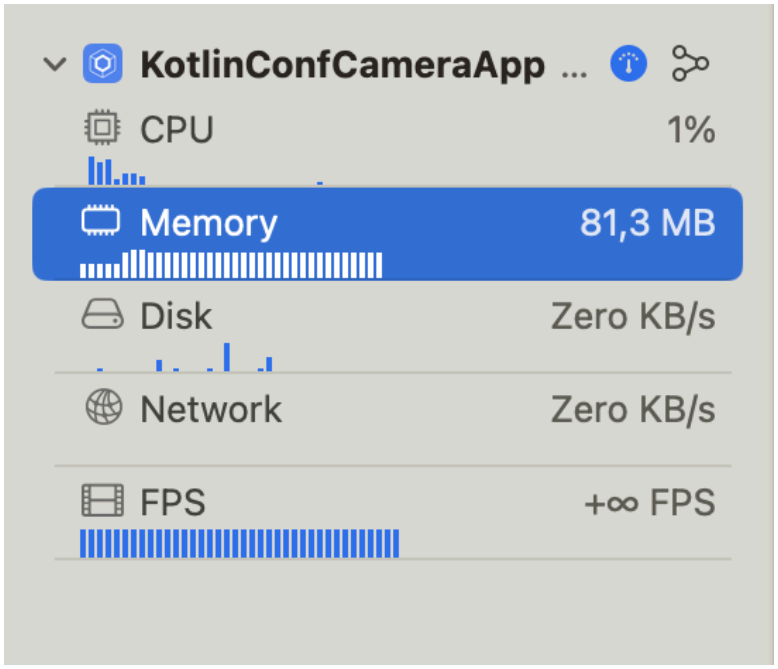
1. Click on the `Run` button at the top left of the screen



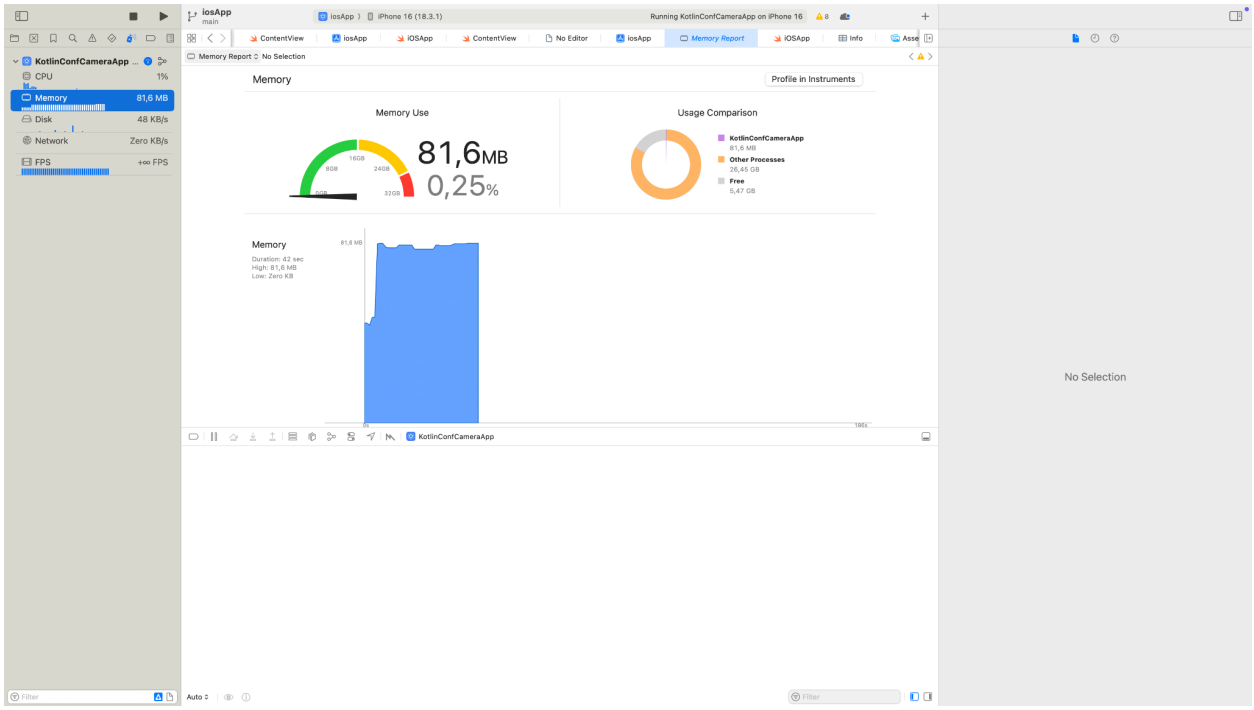
2. The application will then open on the simulator.
3. On the simulator, select an image, the preview screen will display with the image and three buttons.
4. In Xcode, open the `Debug navigator` at the top left icon menu of the screen. To me it looks like a spraypaint can.



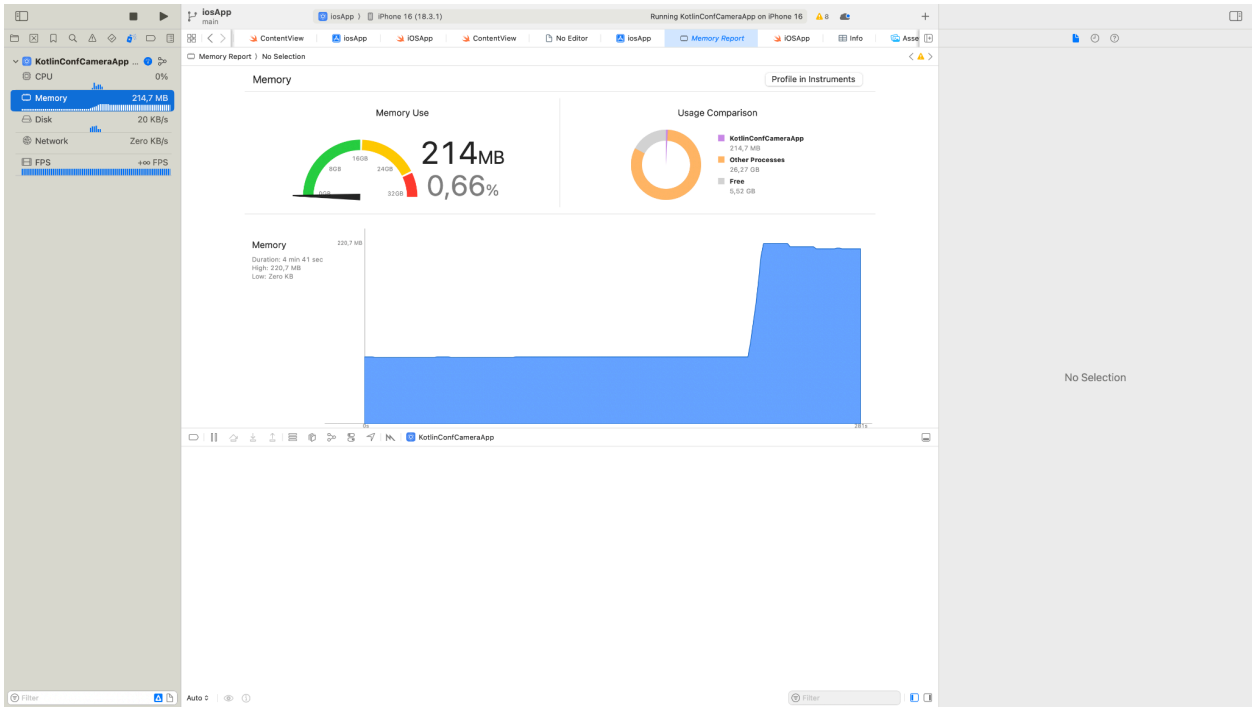
5. Click on the `Memory` section in the `debug navigator`.



6. The screen will then look like this.



7. Take note what the memory usage looks like. Then, on the simulator, press the B&W button a number of times - let's say 10.
8. Back in Xcode, look at the memory usage again. It might look like this, notice the increased memory usage.



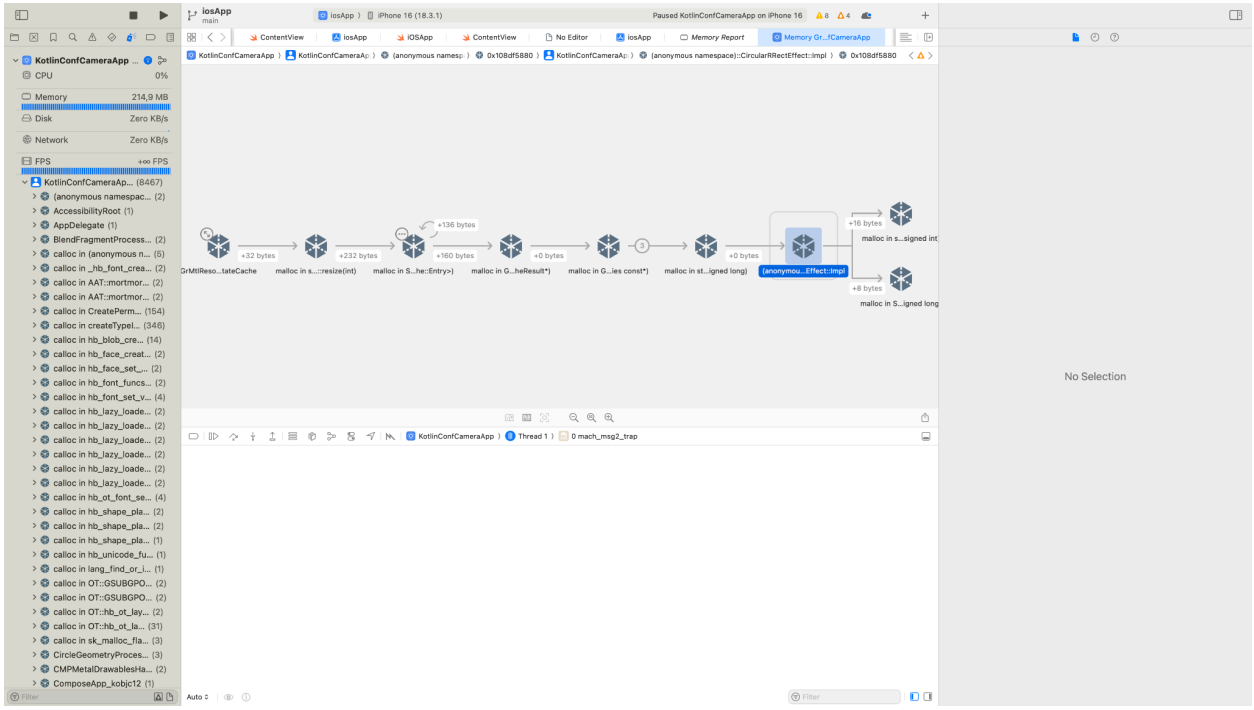
9. We have a memory leak!

Find the cause of the leak

1. Click on the `Debug Memory Graph` button. It's roughly in the middle of the screen, in another icon menu that looks like this.

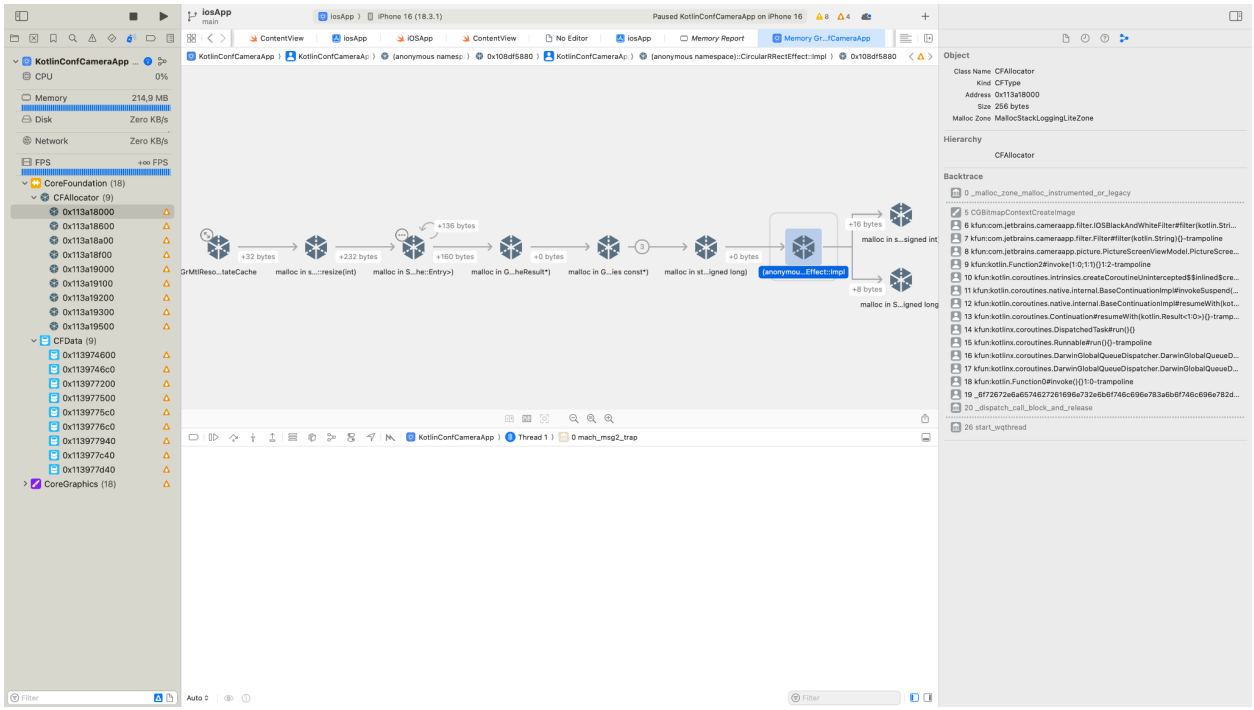
The icon looks like this.

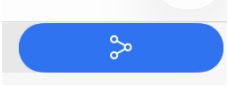
2. The resulting screen looks like this, with a big graph in the middle.

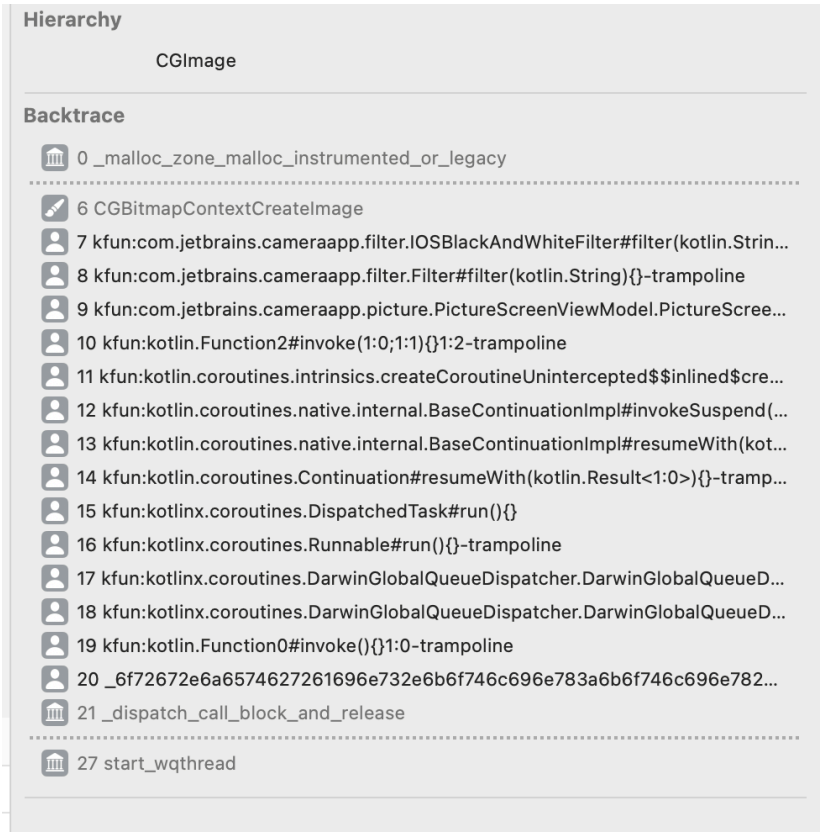


3. At the bottom left of the screen, filter for only leaked allocations by selecting the button at the bottom left of the screen. It looks like this.

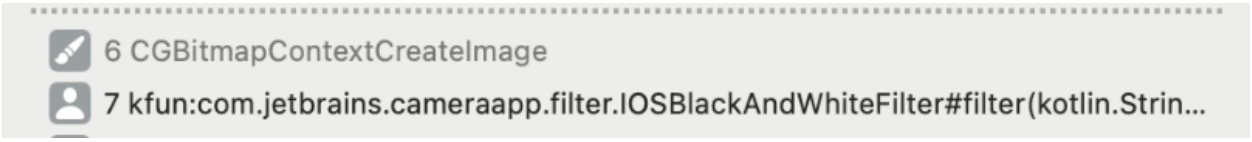
The resulting screen only shows the leaked allocations in the panel on the left.



4. Open up the `CoreGraphics` section, and then `CGImage`. We can see we are leaking `CGImages` somehow. Let's find out where.
5. Click on one of the allocations, then examine the panel on the right. You need to be on the Memory Inspector panel. 



We can see that we are allocating memory for `CGImage` when we call `CGBitmapContextCreateImage`, in the `IOSBlackAndWhiteFilter` class, `filter` function.



Fix the leak and review

1. In Android Studio, open `iosApp/iosApp/src/iosMain/kotlin/com.jetbrains.cameraapp.filter/BlackAndWhiteFilter.ios.kt` in the editor window.
2. Look for the usage of `CGBitmapContextCreateImage`. It should be on line 42, where we create the `grayscaleImage` variable.
3. Oops! We never release the memory! Let's fix that by going to the end of the null-check block and releasing the memory manually.

```
if (grayscaleImage != null) {
    …
    CGImageRelease(grayscaleImage)
```

```
}
```

4. Now, re-run the app and check the memory graph. Should be all fixed with no detected leaks.

How to get the value of a global variable in lldb

1. Get the memory address of the variable

image lookup -r -s imagepath
The memory address is the kfun one.

2. Find the shared library address

image list "KotlinConfCameraApp.debug.dylib"

3. Call the getter function

expr -l objc -O -- ((id (*)())(variablememorypath + appmemoryaddress))()